

**LAPORAN TAKE HOME TASK**  
**PRAKTIKUM PEMROGRAMAN**

**BERORIENTASI OBJEK**

**MODUL 11**



**Disusun Oleh:**

**Ahmad Zaidan 105224044**

**PROGRAM STUDI ILMU KOMPUTER**  
**FAKULTAS SAINS DAN KOMPUTER**  
**UNIVERSITAS PERTAMINA**

**2026**

## I. Pendahuluan

Program Java Express CLI ini adalah implementasi bahasa pemrograman Java yang bertujuan pada konsep Object-Oriented Programming (OOP) lanjutan, terutama dalam modul kali ini *Error and Exception Handling*. Di dalam materi ini terdapat penerapan *custom exception* baik yang bersifat *checked exception* (melalui kelas `Exception`) maupun *unchecked exception* (melalui kelas `RuntimeException`). Program ini dibuat untuk mensimulasikan sistem reservasi dan pemesanan tiket kereta api digital guna mempermudah proses kontrol validitas data penumpang, ketersediaan rute, serta kecukupan kuota kursi secara aman. Dengan demikian, pendahuluan modul ini bertujuan untuk memberikan pemahaman dasar bagaimana struktur try-catch-finally, pelontaran eksepsi (throw dan throws), serta pembuatan kelas pengecualian mandiri bekerja dalam menangani studi kasus nyata tanpa membuat program mengalami *crash* atau berhenti mendadak.

## II. Variabel

| No | Nama Variabel | Tipe Data         | Fungsi                                                                                        |
|----|---------------|-------------------|-----------------------------------------------------------------------------------------------|
| 1  | kodeKereta    | String            | Menyimpan kode unik identitas kereta api.                                                     |
| 2  | namaKereta    | String            | Menyimpan nama dari kereta api                                                                |
| 3  | rute          | String            | Menyimpan informasi rute asal dan tujuan kereta                                               |
| 4  | sisakursi     | int               | Menyimpan kuantitas jumlah kursi yang masih tersedia                                          |
| 5  | daftarKereta  | ArrayList<Kereta> | Sebagai penyimpanan secara dinamis untuk daftar seluruh objek kereta yang terdaftar di sistem |
| 6  | kontrol       | ReservasiKontrol  | Objek pengendali utama untuk mengurus logika bisnis pemesanan dan validasi tiket.             |
| 7  | scanner       | Scanner           | Objek utilitas untuk membaca input data dari pengguna melalui CLI                             |
| 8  | berjalan      | boolean           | Indikator kontrol perulangan ( <i>looping</i> ) menu utama aplikasi.manajemen gudang.         |

### III. Constructor dan method

| No | Nama Metode                           | Jenis metode      | Fungsi                                                                                                                   |
|----|---------------------------------------|-------------------|--------------------------------------------------------------------------------------------------------------------------|
| 1  | Kereta()                              | Constructor       | Menginisialisasi nilai awal objek Kereta (Kode, Nama, Rute, Sisa Kursi) saat pertama kali dibuat.                        |
| 2  | ReservasiKontrol()                    | Constructor       | Menginisialisasi objek ArrayList dan mendaftarkan data kereta awal ke dalam sistem.                                      |
| 3  | DataPenumpangTidakValidException(...) | Constructor       | Membuat objek eksepsi kustom untuk kesalahan format NIK penumpang.                                                       |
| 4  | RuteTidakDitemukanException(...)      | Constructor       | Membuat objek eksepsi kustom untuk kesalahan input kode kereta yang tidak terdaftar.                                     |
| 5  | TiketHabisException(...)              | Constructor       | Membuat objek eksepsi kustom untuk menangani situasi saat jumlah pesanan melebihi sisa kursi.                            |
| 6  | pesanTiket(...)                       | Procedural (void) | Memproses validasi NIK, pencarian kode kereta, pengurangan stok kursi, dan pelemparan eksepsi jika ada data tidak valid. |
| 7  | main(...)                             | Procedural (void) | untuk mengeksekusi menu interaktif CLI dan menangkap berbagai jenis eksepsi.                                             |

## IV. Dokumentasi dan Pembahasan Code

### a. Kereta.java

```
1 public class Kereta {
2     private String kodeKereta;
3     private String namaKereta;
4     private String rute;
5     private int sisaKursi;
6     public Kereta(String kodeKereta, String namaKereta, String rute, int sisaKursi) {
7         this.kodeKereta = kodeKereta;
8         this.namaKereta = namaKereta;
9         this.rute = rute;
10        this.sisaKursi = sisaKursi;
11    }
12
13    Windsurf: Refactor | Explain | Generate Javadoc | X
14    public String getKodeKereta() {
15        return kodeKereta;
16    }
17
18    Windsurf: Refactor | Explain | Generate Javadoc | X
19    public String getNamaKereta() {
20        return namaKereta;
21    }
22
23    Windsurf: Refactor | Explain | Generate Javadoc | X
24    public String getRute() {
25        return rute;
26    }
27
28    Windsurf: Refactor | Explain | Generate Javadoc | X
29    public int getSisaKursi() {
30        return sisaKursi;
31    }
32
33    Windsurf: Refactor | Explain | Generate Javadoc | X
34    public void setSisaKursi(int sisaKursi) {
35        this.sisaKursi = sisaKursi;
36    }
37 }
```

Class ini berfungsi sebagai *blueprint* untuk merepresentasikan objek kereta api di dalam memori program.

→ Pada baris 2 – 5 menggunakan prinsip enkapsulasi dengan mengatur semua atribut menjadi private.

→ Pada baris 6 – 10 itu adalah constructor '**public kereta(...)**' berguna untuk mengisi nilai-nilai karakteristik saat objek kereta pertama kali dibuat di memori.

→ Pada baris 13 – 30 adalah fungsi getter(data yang hanya bisa diakses dari luar kelas) dan setter(unter mengubah nilai-nilainya).

## b. ReservasiKontrol.java

```
1 import java.util.ArrayList;
2
3 public class ReservasiKontrol {
4     private ArrayList<Kereta> daftarKereta = new ArrayList<>();
5
6     public ReservasiKontrol(){
7         daftarKereta.add(new Kereta(kodeKereta: "K01", namaKereta: "Argo Bromo", rute: "JKT - SBY", sisaKursi: 50));
8         daftarKereta.add(new Kereta(kodeKereta: "K02", namaKereta: "Parahyangan", rute: "JKT - BDG", sisaKursi: 15));
9     }
10    public ArrayList<Kereta> getDaftarKereta() {
11        return daftarKereta;
12    }
13    public void pesanTiket(String kode, String nik, String nama, int jumlahTiket)
14        throws RuteTidakDitemukanException, TiketHabisException {
15
16        if (nik.length() != 16) {
17            throw new DataPenumpangTidakValidException(message: "Error: Panjang NIK harus tepat 16 karakter!");
18        }
19
20        if (!nik.matches(regex: "[0-9]+")) {
21            throw new DataPenumpangTidakValidException(message: "Error: NIK harus berupa angka saja, tidak boleh ada huruf!");
22        }
23
24        Kereta keretaDipilih = null;
25        for (Kereta k : daftarKereta) {
26            if (k.getKodeKereta().equalsIgnoreCase(kode)) {
27                keretaDipilih = k;
28                break;
29            }
30        }
31        if (keretaDipilih == null) {
32            throw new RuteTidakDitemukanException("Error: Kode kereta " + kode + " tidak ditemukan.");
33        }
34        if (jumlahTiket > keretaDipilih.getSisaKursi()) {
35            throw new TiketHabisException(namaKereta: "Error: Kursi tidak mencukupi.", keretaDipilih.getNamaKereta(), keretaDipilih.getSisaKursi());
36        }
37
38        keretaDipilih.setSisaKursi(keretaDipilih.getSisaKursi() - jumlahTiket);
39        System.out.println(x: "\n=== RESERVASI BERHASIL ===");
40        System.out.println("Penumpang: " + nama);
41        System.out.println("Kereta : " + keretaDipilih.getNamaKereta());
42        System.out.println("Jumlah : " + jumlahTiket + " tiket");
43    }
44 }
45
46
```

Class ini mengatur penyusunan data awal kereta dan juga menangani semua validasi aturan bisnis pemesanan tiket sebelum stok kursi resmi dikurangi.

→ Di dalam constructor, program akan membuat **ArrayList** dan memasukan 2 objek kereta sesuai soal, yaitu "Argo Bromo" (K01, 50 kursi) dan "Parahyangan" (K02, 15 kursi).

→ program melakukan validasi NIK. Saat method **pesanTiket()** dipanggil, logika pertama adalah memeriksa NIK penumpang. Menggunakan kondisi **nik.length() != 16** dan **!nik.matches("[0-9]+")**. Jika panjangnya salah atau ada huruf, program secara paksa membatalkan proses dengan melempar **DataPenumpangTidakValidException**.

→ pencarian rute juga dilakukan dalam program. Program melakukan *looping* (perulangan) pada **daftarKereta**. Jika kode yang dimasukkan pengguna tidak cocok dengan kereta mana pun di memori (**keretaDipilih == null**), program melempar **RuteTidakDitemukanException**.

→ Program juga melakukan checked exception. Jika kereta ditemukan, sistem mengecek apakah **jumlahTiket > sisaKursi**. Jika kursi tidak cukup, program melempar **TiketHabisException** sambil membawa info nama kereta dan sisa kuota yang ada.

→ jika sudah lolos dari ketiga validasi itu, sisa kursi dikurangkan dan status sukses dicetak.

c. **DataPenumpangTidakValidException.java**

```
1 public class DataPenumpangTidakValidException extends RuntimeException {  
2     public DataPenumpangTidakValidException(String message) {  
3         super(message);  
4     }  
5 }  
6
```

Class ini mewarisi **RuntimeException**.

→Ini digunakan untuk menangani kesalahan murni akibat kesalahan ketik dari pengguna (saat format NIK salah).

d. **RuterTidakDitemukanException.java**

```
1 public class RuteTidakDitemukanException extends Exception {  
2     public RuteTidakDitemukanException(String message) {  
3         super(message);  
4     }  
5 }  
6
```

Class ini mewarisi class utama yaitu **Exception**

→Digunakan untuk mendeteksi kegagalan logis pencarian data pada sistem eksternal (ketika kode kereta tidak terdaftar).

e. **TiketHabisException.java**

```
1 public class TiketHabisException extends Exception {  
2     private String namaKereta;  
3     private int SisaKursi;  
4  
5     public TiketHabisException(String namaKereta, String message, int SisaKursi) {  
6         super(message);  
7         this.namaKereta = namaKereta;  
8         this.SisaKursi = SisaKursi;  
9     }  
10  
11     Windsurf: Refactor | Explain | Generate Javadoc | X  
12     public String getNamaKereta() {  
13         return namaKereta;  
14     }  
15  
16     Windsurf: Refactor | Explain | Generate Javadoc | X  
17     public int getSisaKursi() {  
18         return SisaKursi;  
19     }  
20 }
```

Sama seperti rute, kelas ini mewarisi class **Exception**.

→class ini ditambahkan atribut **namaKereta** dan **SisaKursi** beserta method *getter*-nya. Sehingga, saat error ini ditangkap di kelas utama (**JavaExpressApp.java**), kita tidak hanya menampilkan pesan eror pada umumnya, tetapi bisa juga memberikan informasi detail berupa sisa kursi riil yang tersedia kepada calon pembeli.

#### f. JavaExpressApp.java

```
1 import java.util.InputMismatchException;
2 import java.util.Scanner;
3
4 public class JavaExpressApp {
5     public static void main(String[] mainArgs) {
6         ReservasiKontrol kontrol = new ReservasiKontrol();
7         Scanner scanner = new Scanner(System.in);
8         boolean berjalan = true;
9
10        System.out.println(x: " SELAMAT DATANG DI JAVA EXPRESS CLI ");
11
12        while (berjalan) {
13            System.out.println(x: "\n MENU UTAMA ");
14            System.out.println(x: "1. Lihat Jadwal Kereta");
15            System.out.println(x: "2. Pesan Tiket");
16            System.out.println(x: "3. Keluar");
17            System.out.print(s: "Pilih menu (1-3): ");
18
19            try {
20                int menu = scanner.nextInt();
21                scanner.nextLine();
22
23                switch (menu) {
24                    case 1:
25                        System.out.println(x: "\n== JADWAL KERETA ==");
26                        System.out.printf(format: "%-6s %-15s %-12s %-10s\n", ...args: "KODE", "NAMA", "RUTE", "SISA KURSI");
27                        for (Kereta k : kontrol.getDaftarKereta()) {
28                            System.out.printf(format: "%-6s %-15s %-12s %-10d\n",
29                                k.getKodeKereta(), k.getNamaKereta(), k.getRute(), k.getSisaKursi());
30                        }
31                        break;
32
33                    case 2:
34                        System.out.println(x: "\n--- FORMULIR PEMESANAN ---");
35                        System.out.print(s: "Masukkan Kode Kereta: ");
36                        String kode = scanner.nextLine();
37                        System.out.print(s: "Masukkan NIK: ");
38                        String nik = scanner.nextLine();
39                        System.out.print(s: "Masukkan Nama: ");
40                        String nama = scanner.nextLine();
41                        System.out.print(s: "Jumlah Tiket: ");
42                        int jumlah = scanner.nextInt();
43                        scanner.nextLine();
44
45                        kontrol.pesanTiket(kode, nik, nama, jumlah);
46                        break;
47
48                    case 3:
49                        berjalan = false;
50                        break;
51
52                    default:
53                        System.out.println(x: "Pilihan tidak tersedia. Silakan pilih 1-3.");
54                }
55            } catch (InputMismatchException e) {
56                System.out.println(x: "Error: Input harus berupa angka!");
57                scanner.nextLine();
58            } catch (DataPenumpangTidakValidException e) {
59                System.out.println(e.getMessage());
60            } catch (RuteTidakDitemukanException e) {
61                System.out.println(e.getMessage());
62            } catch (TiketHabisException e) {
63                System.out.println(e.getMessage());
64            } finally {
65                System.out.println("Info: Kereta " + e.getNamaKereta() + " hanya tersisa " + e.getSisaKursi() + " kursi.");
66            }
67            if (!berjalan) {
68                System.out.println(x: "\nTerima kasih telah menggunakan layanan JAVA EXPRESS!");
69                scanner.close();
70            }
71        }
72    }
73 }
74 }
```

File ini sebagai Main dari program, yang berperan sebagai UI berbasis teks yang berhadapan langsung dengan pengguna.

→ Baris 12 – 17 adalah looping menu menggunakan perulangan **while(berjalan)** agar program selalu menampilkan pilihan menu kepada pengguna dan tidak langsung menutup setelah sekali proses.

→ kemudian semua input dari pengguna masuk ke dalam blok **try** sebagai perlindungan.

→ pada bagian blok **catch (InputMismatchException)** berguna untuk menangkap error bawaan Java jika user mengetik huruf saat program meminta angka menu/jumlah tiket. Logika penting di sini adalah pemanggilan `scanner.nextLine()` untuk membersihkan memori *buffer* input agar program tidak terjebak dalam *infinite loop*.

→ pada bagian blok **catch** yang selanjutnya untuk **DataPenumpangTidakValidException**, **RuteTidakDitemukanException**, dan **TiketHabisException** berguna untuk menangkap *custom exception* yang dikirim oleh kelas `ReservasiKontrol`, mengambil pesan error-nya menggunakan `.getMessage()`, lalu menampilkannya secara ramah di layar agar aplikasi tidak *crash*.

→ terakhir adalah blok **finally** berguna sebagai tempat eksekusi di akhir. Logikanya, saat pengguna memilih menu 3 (`berjalan = false`), blok ini akan mendeteksi kondisi tersebut, menampilkan salam perpisahan dari program, dan menutup aliran memori scanner secara aman (`scanner.close()`) untuk mencegah kebocoran memori.



## V. Kesimpulan

Program Java Express CLI ini berhasil membuktikan bagaimana konsep *Error Handling* dapat digunakan untuk membangun aplikasi sistem pemesanan yang ketat dan toleran terhadap kesalahan input (*fault-tolerant*). Melalui simulasi interaktif yang dijalankan, program mampu membedakan penanganan aturan data pengetikan (melalui *unchecked exception*) dengan validasi logika ketersediaan sistem gudang kursi dan rute kereta (melalui *checked exception*). Integrasi blok try-catch-finally yang tepat sangat bagus untuk mengisolasi error yang terjadi di kelas kontrol, menampilkan pesan yang jelas dan ramah kepada pengguna melalui layar UI yang langsung berhadapan kepada pengguna, begitu pula menjamin penutupan aplikasi secara bersih tanpa merusak stabilitas jalannya sistem utama.

#### **IV. Daftar Pustaka**

Tim Asisten Laboratorium Komputer. (2026). Modul Praktikum Pemrograman Berorientasi Objek - Modul 11: Error Handling. Program Studi Ilmu Komputer, Universitas Pertamina

Dosen PBO. (2026). Exception Handling in Java. Universitas Pertamina